

Week-6 Assignment

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Ans:

- **Driver:** The "brain" of the application. It runs the main() function, maintains information about the Spark application, converts the user's code into a physical execution plan (DAG), and schedules tasks to the executors.
- **Cluster Manager:** The resource allocator (e.g., YARN, Kubernetes, or Spark Standalone). It manages the physical machines in the cluster and allocates CPU and memory resources to the application.
- **Executor:** The "workers." These are processes running on the worker nodes that execute the specific tasks assigned by the Driver, process the data, and store intermediate results in memory (caching).

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Ans: Lazy Evaluation means Spark does not do the work immediately when you ask it to .filter() or .select(). Instead, it just takes notes and builds a step-by-step plan (the DAG). It only actually does the work when you ask for a final result (an Action, like .show()). This helps performance because Spark can look at the whole plan in advance and figure out the fastest, most optimized way to do it before starting.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Ans: Implementation Logic: We use inferSchema=True so Spark automatically figures out if a column is a number or text, saving us time.

```
df_q3 = spark.read.csv("week-6_dataset.csv", header=True, inferSchema=True)
```

```
df_q3.printSchema()
```

```
df_q3.show(3)
```

```

root
|-- user_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- old_name: string (nullable = true)
|-- status: string (nullable = true)
|-- amount: double (nullable = true)
|-- base_price: double (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id| category| price| old_name| status| amount|base_price| region|priority|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 8621| 713| Toys|1022.04| Sports Monitor v5| Pending| 5071.2| 1014.24|Midwest| Medium|
| 7024| 720| Sports| 125.7| Clothing Tool v3|Completed| 156.52| 78.26|Midwest| High|
| 5226| 818|Furniture|1758.51|Appliances System v5| Pending|3421.88| 1710.94| West| Medium|
| NULL| 679| Sports| 847.74| Furniture System v3| Canceled|3998.85| 799.77| South| High|
| 9454| 355| Sports|1314.07| Sports Widget v1| Canceled|2601.32| 1300.66| North| High|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans:

- **CSV (Row-based):** Stores data row by row as simple text. To read just one column, Spark still has to scan the whole row, which is very slow for large tables.
- **Parquet (Columnar):** Stores data column by column and compresses it. It is much faster because if you only need 2 columns out of 100, Spark skips the 98 you don't need. This saves massive amounts of time and memory.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans: Implementation Logic: We filter the data first, and then select the columns. This ensures we only process the exact rows we need.

```
from pyspark.sql.functions import col
```

```
df_electronics = (df_source
```

```
    .filter(col("category") == "Electronics")
```

```
    .select("product_id", "price")
```

```
)
```

```
df_electronics.show(5)
```

```
+-----+-----+
|product_id| price|
+-----+-----+
|      376| 261.08|
|      847| 678.07|
|      472| 465.75|
|      815|1611.79|
|      177|1867.66|
+-----+-----+
only showing top 5 rows
```

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

Ans: Implementation Logic: We chain our methods together to rename the column and fix the data type (casting) in one smooth step.

```
from pyspark.sql.types import DoubleType
```

```
df_revised = (df_source
              .withColumnRenamed("old_name", "new_name")
              .withColumn("price", col("price").cast(DoubleType()))
              )

df_revised.select("new_name", "price").printSchema()

df_revised.select("new_name", "price").show(5)
```

```

root
 |-- new_name: string (nullable = true)
 |-- price: double (nullable = true)

+-----+-----+
|          new_name| price|
+-----+-----+
| Sports Monitor v5|1022.04|
| Clothing Tool v3| 125.7|
| Appliances System v5|1758.51|
| Furniture System v3| 847.74|
| Sports Widget v1|1314.07|
+-----+-----+
only showing top 5 rows

```

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Ans: Instead of copying the data everywhere (which takes too much space), Spark just remembers the exact steps (the Lineage Graph) used to create the data. If a worker computer crashes and loses its piece of data, the Driver simply looks at the steps and tells a new worker to recalculate just that missing piece.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans: Implementation Logic: We use the & symbol and put parentheses around each condition so PySpark understands the exact logic

```

df_high_value_completed = df_source.filter(

    (col("status") == "Completed") &

    (col("amount") > 1000)

)

df_high_value_completed.select("product_id", "status", "amount").show(5)

```

product_id	status	amount
134	Completed	7020.25
263	Completed	2920.65
752	Completed	1314.82
472	Completed	1686.56
177	Completed	9232.45

only showing top 5 rows

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Ans: Predicate Pushdown is a smart shortcut. Because Parquet files store extra details (like the highest and lowest numbers in a chunk of data), Spark checks these details *before* reading. If a chunk of data doesn't match your filter, Spark skips reading that chunk entirely. This stops gigabytes of useless data from clogging up the system's RAM.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

Ans: Implementation Logic: We use .withColumn() to easily do math on an existing column and store the answer in a brand new column.

```
df_taxed = df_source.withColumn("final_price", col("base_price") * 1.18)
df_taxed.select("product_id", "base_price", "final_price").show(5)
```

product_id	base_price	final_price
713	1014.24	1196.8032
720	78.26	92.3468
818	1710.94	2018.9092
679	799.77	943.7285999999999
355	1300.66	1534.7788

only showing top 5 rows

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Ans:

- **Transformations:** Lazy operations that take an existing DataFrame and return a new DataFrame. They do not trigger execution; they just add steps to the DAG. (Examples: `.filter()`, `.groupBy()`)
- **Actions:** Eager operations that trigger the actual execution of the DAG. They return a final result to the Driver program or write data out to storage. (Examples: `.show()`, `.count()`)

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Ans: Implementation Logic: This is a mini data pipeline. It reads fast Parquet data, cleans out missing user IDs, and saves it to a simple CSV.

```
(spark.read.parquet("dataset_parquet")
```

```
.filter(col("user_id").isNotNull())
```

```
.write.mode("overwrite").csv("output_cleaned_csv", header=True)
```

```
)
```

```
spark.read.csv("output_cleaned_csv", header=True).select("user_id", "category").show(5)
```

```
+-----+-----+
|user_id| category|
+-----+-----+
|   8621|    Toys|
|   7024|   Sports|
|   5226|Furniture|
|   9454|   Sports|
|   9098| Clothing|
+-----+-----+
only showing top 5 rows
```

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Ans:

- **Client Mode:** The Driver process runs on the local machine where you submitted the application (e.g., your laptop or an edge node). If you disconnect or close your laptop, the application dies. Best for interactive debugging.

- **Cluster Mode:** The Driver process runs inside a container on one of the worker nodes within the cluster. You can submit the job, turn off your laptop, and the job will continue running safely. Standard for production pipelines.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans: Implementation Logic: We use the pipe symbol | to represent "OR" in PySpark so we can capture rows that match either of the two conditions.

```
df_filtered = df_source.filter(  
    (col("region") == "North") |  
    (col("priority") == "High")  
)  
  
df_filtered.select("product_id", "region", "priority").show(5)
```

```
+-----+-----+-----+  
|product_id| region|priority|  
+-----+-----+-----+  
|      720|Midwest|    High|  
|      679|  South|    High|  
|      355|  North|    High|  
|      403|  North|Critical|  
|      376|  South|    High|  
+-----+-----+-----+  
only showing top 5 rows
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Ans: `.collect()` tries to gather *all* the data from the worker nodes and squeeze it into the memory of a single master computer. If you have terabytes of data, this will instantly crash the computer with an Out of Memory (OOM) error. `.show(5)` is safe because it only brings back exactly 5 rows to look at.